Week 7 Reference Sheet: Advanced List Operations

CSC-125-201 | Use this as a reference when completing Homework 7

Homework Topics

- Slicing with step parameter
- Reversing lists
- Sorting: .sort() vs sorted()
- Membership: in / not in
- .index() and .count()
- .insert() and .extend()
- Copying lists
- split() and join()
- Nested lists
- Modify-in-place vs return value

Advanced Slicing

`list[start:end:step]` - step controls how many items to skip.

Expression	Given `n = [0, 10, 20, 30, 40, 50]`	Result
`n[::2]`	Every other item	`[0, 20, 40]`
`n[::3]`	Every third item	`[0, 30]`
`n[1::2]`	Every other, starting at 1	`[10, 30, 50]`
`n[::-1]`	Reversed copy	`[50, 40, 30, 20, 10, 0]`
`n[4:1:-1]`	Backward from index 4 to 2	`[40, 30, 20]`

Sorting

Two approaches with different behavior.

Method	Modifies original?	Returns	Example
`.sort()`	Yes	`None`	`nums.sort()`
`sorted()`	No	New sorted list	`result = sorted(nums)`
`.sort(reverse=True)`	Yes	`None`	`nums.sort(reverse=True)`
`sorted(reverse=True)`	No	New list (descending)	`result = sorted(nums, reverse=True)`

Never write `x = list.sort()` - that stores `None`.

Reversing

Method	Modifies original?	Returns
`[::-1]`	No	New reversed list
`.reverse()`	Yes	`None`

Membership Testing

Expression	Given `m = ["a", "b", "c"]`	Result
`"a" in m`	Is "a" in the list?	`True`
`"z" in m`	Is "z" in the list?	`False`

`"z" not in m`	Is "z" absent?	`True`

Finding and Counting

Method	What it does	Example	Result
`.index(x)`	Position of first x	`["a","b","a"].index("a")`	`0`
`.count(x)`	How many times x appears	`["a","b","a"].count("a")`	`2`

Check with `in` before `.index()` to avoid `ValueError`.

Growing and Combining

Method	What it does	Example	Result
`.insert(i, x)`	Add x at index i	`lst.insert(0, "z")`	`["z", "a", "b"]`
`.extend(list2)`	Add all items from list2	`[1,2].extend([3,4])`	`[1, 2, 3, 4]`
`list1 + list2`	New combined list	`[1,2] + [3,4]`	`[1, 2, 3, 4]`

`.extend()` modifies in place. `+` creates a new list.

Copying Lists

`copy = original` creates a reference, not a copy. Both names point to the same list.

```
safe = original.copy()   # Independent copy
safe = original[:]        # Also independent
safe = list(original)     # Also independent
```

String-List Conversion

Method	Direction	Example	Result
`.split()`	String to list	`"a b c".split()`	`["a", "b", "c"]`
`.split(",")`	String to list	`"a,b,c".split(",")`	`["a", "b", "c"]`
`" ".join(lst)`	List to string	`" ".join(["a","b"])`	`"a b"`
`",".join(lst)`	List to string	`",".join(["a","b"])`	`"a,b"`

`.join()` requires all items to be strings. Convert numbers with `str()` first.

Nested Lists

Lists inside lists. Use double indexing: `grid[row][col]`.

```
grid = [[1, 2], [3, 4], [5, 6]]
row = grid[0]        # [1, 2]
item = grid[0][1]    # 2
```

Modify vs Return Summary

Modifies list (returns None)	Returns a value (list unchanged)
`.append(x)`, `.insert(i, x)`, `.extend(lst)`	`len(lst)`, `sum(lst)`, `max(lst)`, `min(lst)`
`.remove(x)`, `.pop()`, `.clear()`	`sorted(lst)`, `.copy()`

`.sort()`, `.reverse()`	`.index(x)`, `.count(x)`

Common Errors

Error	Cause	Fix
`None` instead of list	`x = lst.sort()`	Use `x = sorted(lst)`
`ValueError`	`.index()` on missing item	Check with `in` first
`TypeError`	`.join()` on non-strings	Convert items with `str()`
Unintended shared changes	`copy = original`	Use `.copy()` or `[:]`
Nested list via append	`lst.append([1,2])`	Use `.extend([1,2])`

Resources

- W3Schools - Python List Methods
- LearnPython.org - Interactive List Tutorial
- Real Python - Split, Concatenate, and Join Strings